EECS 485 Final Exam
Winter 2025 SOLUTION

This is a 120 minute, closed-book exam.

You may use one cheat sheet, 8.5”x11”, double-sided.

No other notes or use of any computing or communication hardware (e.g., calculators, cell phones, etc.) is allowed.

For multiple-choice questions, indicate your response by filling in the appropriate bubble. Fill in the bubble completely, and make sure to erase completely if you change your answer. For free response questions, write your answers clearly in the space provided in each problem.

Collaboration or assistance is prohibited

- You are NOT allowed to collaborate with anyone else to answer these questions.
- You are NOT allowed to solicit assistance from anyone to answer these questions.

You are to abide by the University of Michigan/Engineering Honor Code. To receive a grade, please sign below to signify that you have kept the honor code pledge.

I have neither given nor received aid on this exam, nor have I concealed any violations of the Honor Code.

Signature: _____

Name: _____

Uniqname: _____

UMID: _____

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Multiple Choice (40 points)

Demo Question: How should I fill in my responses for a Multiple Choice question?

- **A. Fill in the chosen bubble completely and only select ONE per question!**

1. A Worker in a distributed system is sending periodic heartbeats to a Manager process on **TCP**. The network becomes congested, and some heartbeat messages are delayed. What is the most likely outcome?

◯ A. The Manager receives the heartbeats in the wrong order but still processes them.
◯ B. The Worker automatically switches to UDP.
◯ C. The Manager assumes the Worker is dead.
◯ D. The Worker queues all heartbeats and sends them in bursts.
◯ E. The heartbeats are dropped, and the Worker is disconnected from the network.

Solution: A is wrong because TCP guarantees in-order delivery. B is wrong because the Worker can't just automatically switch to UDP. D is wrong because sending heartbeats in bursts would not help the Manager determine if the Worker is alive at a particular time. E is wrong because TCP is reliable and guarantees packets will be delivered eventually. C is correct because the heartbeats could be dropped due to congestion and by the time the packet is resent, the Manager may have already marked the Worker as dead. TCP's reliability mechanisms make it slower than UDP.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

2. Why is context switching ("taking turns") slower for processes than threads? Choose the best answer.

◯ A. Switching processes require saving and restoring more state.
◯ B. Threads must always run on separate CPU cores.
◯ C. Context switching only happens when a process explicitly requests it.
◯ D. Threads require more overhead than processes to switch.
◯ E. Processes and threads have identical memory access patterns to shared memory.

Solution: B is incorrect because threads can run on the same or different cores, and scheduling is managed by the OS. C is incorrect because context switches can be forced by the OS. D is incorrect because process switches require more overhead than threads, not the other way around. E is incorrect because processes have separate memory spaces, while threads share memory. A is correct because process context switches require saving/restoring entire address spaces, whereas threads share memory.

3. Which of the following is TRUE about TCP congestion and flow control?

◯ A. The number of the sender's packets determines the size of the receiver's sliding window
◯ B. The TCP congestion window should increase when a packet is dropped
◯ C. The TCP congestion window should increase when the sender receives an ACK
◯ D. If the sliding window is working properly, and the sender has a lot of data to to send, the receiver's queue will usually be about 65% full
◯ E. Both C and D are correct

Solution: C - the congestion window can increase when the sender receives an ACK because if the packet got there safely, there's room to send more. D is incorrect because the queue should usually be almost full.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

4. Which of the following is a VALID output of the following code snippet?

```
shutdown = False
def main():
      t1 = threading.Thread(target=worker1)
      t1.start()
      t2 = threading.Thread(target=worker2)
      t2.start()
      t1.join()
      print("worker1 joined")
      t2.join()
      print("worker2 joined")

      print(counter)
if __name__ == "__main__":
     main()
```

```
def worker1():
     time.sleep(0.1)
     global shutdown
     shutdown = True
     print("worker1 done")


def worker2():
     while not shutdown:
         time.sleep(0.1)
         print("worker2 done")
```

◯ A. worker1 done, worker1 joined, worker2 done, worker2 joined

◯ B. worker1 joined, worker1 done, worker2 joined, worker2 done

◯ C. worker1 done, worker1 joined

◯ D. worker1 done

◯ E. None of the above will print, as the second join will block the program indefinitely

Solution: A, the code will run correctly as thread 1 updates shutdown to unblock thread 2 before the main thread joins thread 2.

Note: During the exam, we noticed that the variable counter (printed in the last line of main) was never defined. This line was kept in by mistake and it had no effect on the solution.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

5. Which of the following is FALSE about how network issues are handled?

◯ A. If packets are out of order, UDP allows applications to handle their order

◯ B. If packets disappear, UDP resends lost packets

◯ C. If packets arrive out of order, the TCP buffers reassemble packets using sequence numbers

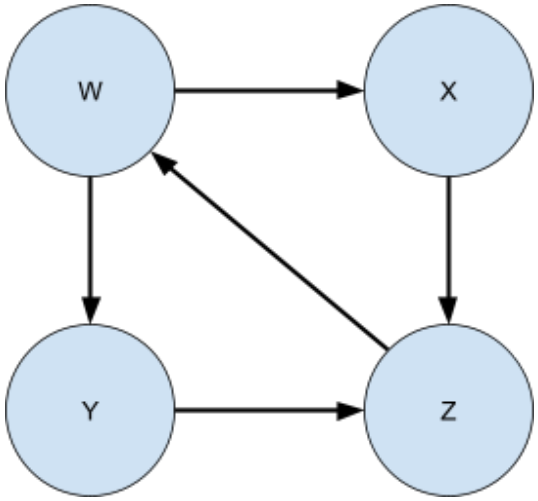
◯ D. If packets are repeated, the TCP receiver ignores duplicate sequence numbers

◯ E. If a set of senders are overloading the network, the TCP congestion control system uses a congestion window to limit the number of packets

Solution: B - UDP follows the better never than late method, and ignores lost packets

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____



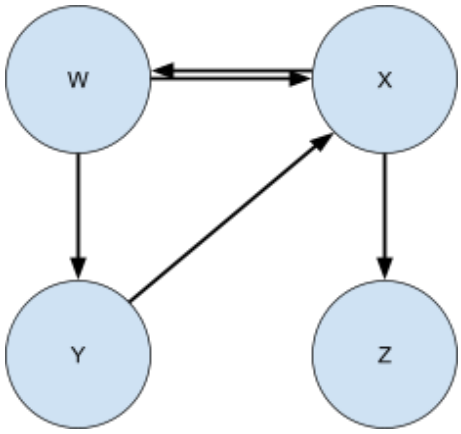
6. Based on the diagram above and using a typical damping factor of 0.85, which node will have the highest PageRank score after three iterations?

- ◯ A. W
- ◯ B. X
- ◯ C. Y
- ◯ D. Z
- ◯ E. None of the above

Solution: The damping factor is 0.85, so each node distributes 85% of its PageRank to its outbound links, and 15% is distributed evenly across all nodes. Z receives links from both X and Y and redistributes its PageRank back to W. Since PageRank is iterative, nodes with more incoming links (especially from well-connected nodes) tend to have a higher score. Z gets PageRank from both X and Y, which helps it gain a higher score.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____



7. Using the diagram above, say we know the following ranks after applying the PageRank algorithm: W ranks 2nd, X ranks 1st, Y ranks 3rd, and Z ranks 4th. Which of the following changes would most likely result in X having the lowest PageRank score of any node?

◯ A. Adding a link from X → Y
◯ B. Removing the link from X → Z
◯ C. Adding a link from Z → X
◯ D. Removing the link from W → X
◯ E. Adding a link from X → W

Solution:

A is incorrect because this would create an additional link from X to Y. Since X is already the highest-ranked node, this would distribute some of X's score to Y, but it's unlikely to drastically reduce X's score to the lowest position.

B is incorrect because this would reduce X's contribution to Z, which would reduce Z's PageRank. However, X still links to W and Y.

C is incorrect because this would directly increase X's authority score because adding an incoming link from Z would increase the authority flowing into X, increasing X's PageRank.

D is correct because this would decrease X's PageRank significantly as W is an important node with a high rank, and X benefits from its incoming link. Removing this link would be major in lowering X's PageRank.

E is incorrect because adding an outgoing link from X to W would increase the number of links from X, but W already has a strong rank, so this won't have much effect on X's rank.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

8. Which of the following is FALSE about calculating the PageRank of a webpage?

◯ A. Being linked by many pages can increase the PageRank of a webpage.
◯ B. Repeating the keyword multiple times can increase the PageRank of a webpage.
◯ C. Linking to many pages may not increase the PageRank of a webpage.
◯ D. PageRank can measure the importance of a page.
◯ E. Adding the damping factor can avoid the sink nodes/region situation.

Solution:

A) True, linked by many pages can increase the pagerank.
B) False, term frequency has nothing to do with pagerank.
C) True, it needs to be linked by many important pages.
D) True, it's the definition of pagerank.
E) True, the damping factor (1/d)/N will avoid the sink node siatuation.

9. Which of the following is FALSE about normalized TF-IDF and Vector Space?

◯ A. TF-IDF is good when documents are disjointed and are of high quality.
◯ B. The more a word occurs in the document, the higher the TF-IDF.
◯ C. Lower document frequency may lead to a higher TF-IDF.
◯ D. In order to not give longer documents more weight, we normalize the TD-IDF.
◯ E. If two documents are similar, the angle between them will be smaller, and the cosine will be larger.

Solution:

A) True.
B) False, there are a lot of counterexamples. For example, the lecture kangaroo example. We can normalize it to vector length to balance the weight.
C) True.
D) True, the kangaroo example in the lecture.
E) True, according to the definition of cosine similarity.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

10. Which of the following is TRUE about Search?

◯ A. Crawlers are used to build the link graph where each website is a directed edge and each hyperlink is a vertex.

◯ B. Crawlers can crawl all web pages that exist.

◯ C. A surface crawler that has no credentials can crawl the 485 lecture material.

◯ D. If a page links many other web pages, it's important.

◯ E. When there is a new query, the pagerank score does not need to be recalculated.

Solution:

A) False, because website is a vertex and hyperlinks are the directed edges.
B) False, they can't crawl those pages that are on robot.txt.
C) False, surface crawlers can't crawl password-protected pages.
D) False, page is important if it's linked by many important pages.
E) True, PageRank is not query-dependent.

11. A machine dies in a search engine with a segmented inverted index. Which could be true?

◯ A. Segment-by-term is used: when users search for "Britney Spears," they receive no results.

◯ B. Segment-by-term is used: when users search for "eecs485," the eecs485 page is no longer in the results.

◯ C. Segment-by-document is used: when users search for "Britney Spears," they receive no results.

◯ D. Segment-by-document is used: when users search for "eecs485," the eecs485 page is no longer in the results.

◯ E. A and D

Solution: E. A and D

If you use segment-by-term, then you will lose certain queries when a storage machine goes down. Therefore, if you use segment-by-term, A is possible. If you use segment-by-document, then you will lose some search results when a storage machine goes down. Therefore, if you use segment-by-document, D is possible.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

12. Find the Jaccard Similarity between the following documents using 2-word-shingles

Document A: “Ellen’s favorite project is Project 4.”

Document B: “Hailey’s favorite project is Project 5.”

- ◯ A. 0
- ◯ B. 2/7
- ◯ C. 3/7
- ◯ D. 4/7
- ◯ E. 1

Solution: C. 3/7

Document A Shingles: {“Ellen’s favorite,” “favorite project,” “project is”, “is Project,” “Project 4”}

Document B Shingles: {“Hailey’s favorite,” “favorite project,” “project is,” “is Project,” “Project 5”}

There are 3 common shingles out of 7 total unique shingles. Therefore, the Jaccard Similarity is 3/7.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

13. In anticipation of Taylor Swift's next tour, Jenna is creating an online market for honorable ticket resellers. In the case of a network database failure, should she prioritize consistency or availability and why?

◯ A. Consistency to ensure that all ticket information presented to a customer is correct.
◯ B. Consistency to ensure that each customer's ticket payment is authorized by their credit card company.
◯ C. Availability to ensure that the website can always be reached by customers.
◯ D. Availability to ensure that ticket information is always presented to customers even if the information is outdated.
◯ E. None of the explanations are correct.

Solution: A

A. Correct: You should prioritize consistency because very bad things could happen if stale information is presented to customers (such as multiple customers buying the same tickets). Consistency ensures that all data returned is correct or an error is returned if data is not reachable.
B. Wrong: Consistency is not related to authorizing payment methods.
C. Wrong: Availability will impact whether the database information is reachable rather than impacting the entire website.
D. Wrong: Very bad things could happen if outdated data is presented to customers (such as a customer buying a ticket that is no longer available)
E. Wrong: because A is correct

14. Which usage pairing(s) is correct?

◯ A. Containerization: Machine's capacity should be fully utilized
◯ B. Containerization: Application's data needs to be saved for later
◯ C. Virtualization: Application's OS is different
◯ D. Virtualization: Application's user count fluctuates quickly
◯ E. Both A and C are correct

Solution: E. To fully utilize a machine's capacity, we can run multiple containers (A). Different OSes require virtualization (C). B is incorrect as containerization doesn't allow for persistent data storage, D is incorrect as virtualization startup is slow and won't handle fluctuating user counts.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

15. You browse to cse.engin.umich.edu. Which of the following is true? Assume caching with an empty starting cache.
    - ◯ A. Immediately reloading the page will require the .umich nameserver to be re-queried
    - ◯ B. The browser will query the .edu nameserver
    - ◯ C. A subsequent access to me.engin.umich.edu will require fewer network requests
    - ◯ D. The .com nameserver will be cached
    - ◯ E. Both B and C are true

Solution: C

A. Since caching is present, cse.engin.umich.edu's address will be saved in the cache and the .umich nameserver will not need to be re-queried as part of the resolution process
B. The browser has an empty cache so the DNS resolution will be passed to the DNS resolver. The DNS resolver will be in charge of querying the .edu nameserver
C. The engin.umich.edu nameserver would be saved in cache meaning that umich.edu, .edu and root nameserver would not need to be re-queried
D. The .com nameserver is not part of the resolution path so won't be cached

16. Which of the following best distinguishes content-based collaborative filtering from user-based collaborative filtering in recommendation systems?
    - ◯ A. Content-based needs lots of users; user-based works with few.
    - ◯ B. Content-based is better for popular items; user-based is better for new items.
    - ◯ C. Content-based suggests similar items to what a user liked before; user-based suggests what similar users liked.
    - ◯ D. Content-based suffers from the cold-start problem; user-based does not.
    - ◯ E. Content-based uses other users' ratings; user-based uses item features.

Solution: Content-based filtering focuses on the user's past behavior and item features ("You liked this, so you'll like that"). Collaborative filtering relies on similar users ("People like you liked this"). On the exam taken at regular time, options B and D (D is now rewritten) were the same and were graded the same..

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

17. Which of the following changes would best reduce the environmental impact of cryptocurrency mining?

◯ A. Encourage miners to join mining pools to combine their computational power which increases their chances of successfully mining a block.
◯ B. Encourage miners to relocate to low cost of living areas.
◯ C. Decreasing the reward offered for winning a block.
◯ D. Switching from a proof-of-work model to a proof-of-stake model.
◯ E. None of the above would reduce environmental impact.

Solution: A is incorrect because the same level energy is still being consumed by each individual miner in the pool having no net effect on the environmental impact. B is incorrect because it does not relate to energy consumption. C is incorrect because decreasing the reward for winning a block could cause miners to leave (thus reducing the environmental impact) but this is not a guarantee, since it could also cause miners to increase their computational power so that they win blocks at a higher rate in order to maintain their prior profits rate. D is correct because Proof-of-work requires miners to solve complex mathematical problems, consuming significant amounts of electricity and contributing to environmental concerns, however proof-of-stake has miners essentially bid to win the block rather than solve a puzzle significantly minimizing energy consumption.

18. What does a single TOR node know?

◯ A. an input node knows the destination
◯ B. an exit node knows the source and destination
◯ C. an input node knows the source and destination
◯ D. an exit node knows the destination
◯ E. Both input and exit nodes know the source and destination

Solution: D is correct, in Tor an input node knows the source and an exit node knows the destination.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

19. What is true about Ethereum smart contracts?

- ◯ A. They can be reversed and fixed after release
- ◯ B. Miners control them and are able to change the contract terms
- ◯ C. Any vulnerability can be exploited permanently
- ◯ D. Contracts require permission from a central authority to run
- ◯ E. Ethereum does not use smart contracts NFTs do

Solution: C is true because once the code is released it is cannot be changed. Meaning any existing bugs in the code that can be exploited can be exploited permanently.

20. Akanksha goes to a sealed-bid second-price auction and brings her minion for good luck. Akanksha bids $50, her minion bids $75, and Gru bids $60. What is the possible outcome of this round?

- ◯ A. Akanksha re-bids $80
- ◯ B. Akanksha's minion wins and pays $75
- ◯ C. Gru's proxy rebids $80 for him
- ◯ D. Gru wins and pays $60
- ◯ E. Akanksha's minion wins and pays $60

Solution: E is correct in a sealed-bid second-price auction the bidders submit their bids at once and the highest bid wins but pays the second-highest bid price.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Free Response 1: Gru(te) Force 485 (30 points)

Gru is helping EECS 485 staff crack stolen hashed passwords using a `MinionServer` that coordinates brute-force password cracking jobs and reports plaintext passwords. Gru communicates with the `MinionServer`, sending new jobs and shutdown messages. Password cracking can be very CPU intensive – use `Minion` processes to divide and conquer. Each minion process is created with a hashed password and a subset of possible guesses that it exhaustively checks.

`MinionServer` Process

- **Main Thread**
  - Starts a thread for TCP listening.
  - Loops until shutdown, periodically checking if there are any new password jobs.
  - Once a new job is started, create the specified number of Minion **processes**. The job message contains a list of lists, each list corresponds to the guesses to be assigned to a minion. Length of list will always equal the number of minions.
  - Ensure all minions working on previous jobs have terminated before starting a new job, and only process one job at a time.
- **TCP Thread**
  - Message type "`new_job`": Update the job queue with the new job details.
  - Message type "shutdown": Update the relevant variables to terminate the server and notify all minions.
  - Message type "`woohoo`": Send the plaintext password to Gru and notify all minions to terminate.
  - Message type "`failure`": Update count of number of minion tasks failed. Notify all minions to terminate if all minions failed.

Creating a new process

Similar to the multithreading library from project 4, we can use the multiprocessing library to spawn new processes:

```
>> p = Process(target=<function>, args=(<arg1>,<arg2>,...))
>> p.start()
>> p.join()
```

Assigning minion ports

Each process should have the same host (as the `MinionServer`), but a different port.
For example, `MinionServer` port = 8000, `Minion` ports = 8001, 8002, …

Once a shutdown message is received, **all threads must terminate** before the program ends.

Avoid busy waiting wherever possible.

Simplifying Assumptions

- Don't need to implement fault tolerance.
- All messages are in the correct form and no TCP errors can occur.
- `tcp_client()` will retry sending a message until it is received

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Provided helper functions

You can use these functions in your implementation. Note that these are **not** class members.

```
def tcp_client(host, port, msg):
"""
Creates a socket connection to (host, port) and sends the msg dictionary.
"""
def tcp_server(host, port, handler, signals):
"""
Listens for TCP connections on (host, port) and receives messages until
signals["shutdown"] is set to True. When a message is received, handler(msg)
is called, where msg is a dictionary.
"""
def hash(plaintext):
"""
Takes in a string plaintext and returns the hashed int value.
"""
```

Message Descriptions

`MinionServer` **receives** the following messages using TCP

From Gru:
`{`
`'type': "new_job",`
`'num_minions': <int>,`
`'hashed_pw': <string>,`
`'guesses': <list<list<string>>>`
`}`
	From a `minion`:
	`{`
	`'type': "woohoo",`
	`'password': <string>`
	`}`

From Gru (to be forwarded to minions):
`{`
`'type': "shutdown"`
`}`
	From a `minion`:
	`{`
	`'type': "failure"`
	`}`

`MinionServer` **sends** the following messages using TCP

To Gru:
`{`
`'password': <string>`
`}`
	To a minion:
	`{`
	`'type': "shutdown"`
	`}`

```
class MinionServer:
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
def __init__(self, host: str, port: int):
    # Must only use the following data structures
    self.signals = {"shutdown": False}   # shutdown signal
    self.minions = {}                    # {(host, port): process, ...}
    self.job_queue = []
    self.num_failures = 0
    self.current_job = {}
    self.gru_host = ...                  # already initialized
    self.gru_port = ...                  # already initialized
```

```
    tcp_thread = threading.Thread(
                    target=tcp_server,
                    args=[host, port,  self.handle_tcp, self.signals])
    tcp_thread.start()

    while not self.signals["shutdown"]:
         if len(self.job_queue) > 0:
             # make sure previous job processes have terminated
             for process in self.minions.values():
                 process.join()
             self.minions = {}
             self.current_job = self.job_queue.pop(0)
             self.num_failures = 0

             for i in range(self.current_job["num_minions"]):
                 minion_port = port + i + 1
                 self.minions[(host, minion_port)] =
                           Process(target=self.start_minion,
                           args=[host, minion_port,
                           self.current_job["hashed_pw"],
                           self.current_job["guesses"][i]])
                 self.minions[(host, minion_port)].start()
         time.sleep(0.1)

     tcp_thread.join()
     for process in self.minions.values():
             process.join()
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
def start_minion(self, host, port, hash_password, guesses):
    # Creates a new instance of the Minion class.
    Minion(host, port, hash_password, guesses)

def handle_tcp(self, msg):
```

```
        if msg["type"] == "new_job":
            self.job_queue.append(msg)
            return
        if msg["type"] == "shutdown":
            self.signals["shutdown"] = True
        elif msg["type"] == "woohoo":
            tcp_client(self.gru_host,
                       self.gru_port,
                       {"password": msg["password"]})
        elif msg["type"] == "failure":
            self.num_failures += 1
            if self.num_failures != self.current_job["num_minions"]:
                return
        for minion, _ in self.minions.items():
            tcp_client(minion[0], minion[1], {"type": "shutdown"})
```

Minion Process

This process should keep running until a shutdown message is received.

- **Main Thread**
  - Starts a thread for TCP listening.
  - Loops through all guesses in the minion's input, comparing each guess' hash against the hashed target password.
  - If a match is found, sends the correct guess back to the `MinionServer`.
  - If no match is found in the guesses, sends failure message to the `MinionServer`.
- **TCP Thread**
  - Message type "shutdown": Update the relevant variable

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
class Minion:

    def __init__(self, host: str, port: int, minion_server_host: str,
minion_server_port: int, hash_password: str, guesses: list):
        self.signals = {“shutdown”: False}      # shutdown signal
        tcp_thread = threading.Thread(
                        target=tcp_server,
                        args=[host, port,  self.handle_tcp, self.signals])
```

```
        tcp_thread.start()

        for guess in guesses:
            if hash_password == hash(guess):
                tcp_client(minion_server_host,
                           minion_server_port,
                           {"type": "woohoo",
                            "password": guess})
                tcp_thread.join()
                return

        tcp_client(minion_server_host,
                   minion_server_port,
                   {"type": "failure"})
        tcp_thread.join()
```

```
     def handle_tcp(self, msg):
```

```
        if msg["type"] == "shutdown":
            self.signals[“shutdown”] = True
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Free Response 2: LOGGER.info (30 points)

eecs485staff have partnered with cloud providers to track service performance! Write a **two-stage MapReduce pipeline** that identifies error statistics in server log files.

Input
map1.py will receive log file entries from different services in the following format:

`<timestamp> <request_id> <service_name> <event_type>`

Example input:
```
1742489590 5589 nginx request
1742489590 1729 gunicorn request
1742489595 0443 postgresql response
1742489590 0443 postgresql request
1742489613 1729 gunicorn response
1742489632 5589 nginx response
1742489232 1583 nginx request
1742489732 1583 nginx response
```

`timestamp`: modified unix timestamp in milliseconds.

`request_id`: unique client request.

`service_name:` only one service can be requested per `request_id`.

`event_type`: a `request` indicates a service has begun processing and a `response` indicates it has finished processing.

Stage 1
The first stage must determine if a service call is an **error**, meaning either the `request` or `response` is missing. Calculate the **latency** between the `response` and `request` for each non-error service call (request to response).

Hint: Latency is defined as the time between receiving a request and a response

Hint: You should pass along whether the service call was missing a `request` or `response` so the next stage can distinguish the type of error per service call.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Stage 2

The second stage must report all errors and identify any **slow** non-error service calls, which are defined as those with a latency **>= 2 * `average_latency`** amongst those calls for that service.

Output

For each service, display the `service_name` and the `average_latency`, followed by a list of errors and slow service calls. If a call is an error, print whether the `request` or `response` is missing.

```
<service_name> <avg_latency>ms
      <request_id_i> – missing <request|response>
      <request_id_j> - slow
      …
<service_name> <avg_latency>
      …
```

Example output:

```
mongodb 400ms
      0987 - missing request
      5678 – slow
      1234 - missing response
gunicorn 500ms
      …
```

Constraints

Similar to how you implemented the pipeline in Project 5, write your Map and Reduce code in Python3 using the Hadoop Streaming Interface with key-value pairs separated by a tab (`\t`) character. Read all input from `stdin` and write all output to `stdout`.

1. You must meaningfully use all outlined stages to receive credit.
2. You must use **itertools.groupby** to implement the reduce stages.
3. All Map stages must use **O(1) memory** to receive credit.
4. All Map stages must produce **multiple groups** to receive credit.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
#!/usr/bin/env python3
# map1.py
# Assume all packages are imported
```

```
# Input: <timestamp> <request_id> <service_name> <event_type>
# Output: <request_id>\t<service_name> <timestamp> <event_type>

for line in sys.stdin:
    timestamp, request_id, service_name, event_type = line.strip().split()
    print(f"{request_id}\t{service_name} {timestamp} {event_type}")
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
#!/usr/bin/env python3
# reduce1.py
# Assume all packages are imported

def reduce_1_group(key, group):
    # Input: <request_id>\t<service_name> <timestamp> <event_type>
    # Output: <request_id> <service_name> <latency>
    # Output: <request_id> <service_name> error: <error_type>

    request_time, response_time, name = None, None, None
    for line in group:
        _, parts = line.strip().split('\t')
        service_name, timestamp, event_type = parts.split()
        name = service_name

        # record the response and request timestamps
        if event_type == 'request':
            request_time = int(timestamp)
        else:
            response_time = int(timestamp)

    if request_time is None or response_time is None:
        missing_event = 'request' if request_time is None else 'response'
        print(f'{key} {name} error: {missing_event}')
    else:
        latency = response_time - request_time
        print(f'{key} {name} {latency}')

def keyfunc(line):
    return line.partition("\t")[0]

def main():
    for key, group in itertools.groupby(sys.stdin, keyfunc):
        reduce_1_group(key, group)

if __name__ == "__main__":
    main()
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
#!/usr/bin/env python3
# map2.py
# Assume all packages are imported
```

```
# Input: <request_id> <service_name> <latency>
# Input: <request_id> <service_name> error: <event_type>
# Output: <service_name>\t<request_id> <latency>
# Output: <service_name>\t<request_id> error: <error_type>

for line in sys.stdin:
    line = line.strip().split()
    request_id, service_name = line[0], line[1]

    if len(line) == 3:
        latency = line[2]
        print(f'{service_name}\t{request_id} {latency}')
    else:
        error_type = line[3]
        print(f'{service_name}\t{request_id} error: {error_type}')
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
#!/usr/bin/env python3
# reduce2.py
# Assume all packages are imported

def reduce_2_group(key, group):
```

```
# Input: <service_name>\t<request_id> <latency>
# Input: <service_name>\t<request_id> error: <error_type>
# Output: <service_name> <avg_latency>
              <request_id_i> - error: <error_type>
              <request_id_j>

ids, output, latencies = [], [], []
for line in group:
    _, parts = line.strip().split('\t')
    data = parts.split()

    if 'error' in data:
        error_type = data[-1]
        output.append(f'{data[0]} - missing {error_type}')
    else:
        ids.append(data[0])
        latencies.append(int(data[-1]))

avg_latecy = sum(latencies) / len(latencies)
for i in range(len(ids)):
    if latencies[i] >= 2 * avg_latency:
        output.append(f'{ids[i]} - slow')

print({key} {avg_latency}ms)
for call in output:
    print(f'\t{call}')
```

```
def keyfunc(line):
    return line.partition("\t")[0]

def main():
    for key, group in itertools.groupby(sys.stdin, keyfunc):
        reduce_2_group(key, group)

if __name__ == "__main__":
    main()
```


Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/